Markov Chain Analysis of Snakes and Ladders

Student name: Mia Emanuele

Table of contents

Introduction

Markov Chains provide a mathematical framework for modelling stochastic systems whose future state depends only on their current state. Snakes and Ladders provides an intuitive example because each move is governed by probability while snakes and ladders alter the transition between states. This report models the game as an absorbing Markov Chain to investigate transition probabilities, expected absorption times and the probability of completing the game without returning to the starting square.

Task 1 – Markov Chains

1.1 Transition matrix P

To model the Snakes and Ladders game as a Markov Chain, a transition matrix *P*was constructed. Each element of the matrix represents the probability of moving from one board position (state) to another following a single coin flip while accounting for the effects of snakes and ladders. Consequently, the transition matrix provides a complete probabilistic description of the game's dynamics and forms the foundation for all subsequent analysis.

```
import numpy as np

P = np.array([

# (present state) from 0 1 2 3 4 5 6 7 8                    (next state)

              [0.0, 0.0, 0.0, 0.5, 0.5, 0.0, 0.0, 0.0, 0.0], # to 0

              [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0], # to 1

              [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0], # to 2

              [0.0, 0.5, 0.5, 0.0, 0.0, 0.5, 0.5, 0.0, 0.0], # to 3

              [0.5, 0.5, 0.5, 0.5, 0.0, 0.0, 0.0, 0.0, 0.0], # to 4

              [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0], # to 5

              [0.5, 0.0, 0.0, 0.0, 0.5, 0.5, 0.0, 0.0, 0.0], # to 6

              [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0], # to 7

              [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.5, 1.0, 1.0]  # to 8

                                                          ])

print(P)
```

The resulting transition matrix is shown below.

```
[[0. 0. 0. 0.5 0.5 0. 0. 0. 0. ]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. ]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. ]
 [0. 0.5 0.5 0. 0. 0.5 0.5 0. 0. ]
 [0.5 0.5 0.5 0.5 0. 0. 0. 0. 0. ]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. ]
 [0.5 0. 0. 0. 0.5 0.5 0. 0. 0. ]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. ]
 [0. 0. 0. 0. 0. 0. 0.5 1. 1. ]]
```

1.2 The expected number of coin flips needed to reach square k from square i

To calculate the expected number of coin flips required to reach the absorbing state, it was first necessary to construct the transient-state transition matrix N. This matrix contains only the transient (non-absorbing) states of the Markov Chain and is required when calculating the fundamental matrix used in absorbing Markov Chain analysis.

```
N = np.array([
# (present state) from 0 1 2 3 4 5 6 7 8            (next state)
              [0.0, 0.0, 0.0, 0.5, 0.5, 0.0, 0.0, 0.0], # to 0
              [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0], # to 1
              [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0], # to 2
              [0.0, 0.5, 0.5, 0.0, 0.0, 0.5, 0.5, 0.0], # to 3
```

```
                [0.5, 0.5, 0.5, 0.5, 0.0, 0.0, 0.0, 0.0], # to 4
                [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0], # to 5
                [0.5, 0.0, 0.0, 0.0, 0.5, 0.5, 0.0, 0.0], # to 6
                [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0] # to 7
                                                        ])
print(N)
```

The resulting transient-state transition matrix N is shown below.

```
[[0. 0. 0. 0.5 0.5 0. 0. 0. ]
[0. 0. 0. 0. 0. 0. 0. 0. ]
[0. 0. 0. 0. 0. 0. 0. 0. ]
[0. 0.5 0.5 0. 0. 0.5 0.5 0. ]
[0.5 0.5 0.5 0.5 0. 0. 0. 0. ]
[0. 0. 0. 0. 0. 0. 0. 0. ]
[0.5 0. 0. 0. 0.5 0.5 0. 0. ]
[0. 0. 0. 0. 0. 0. 0. 0. ]]
```

An identity matrix of the same dimensions as $N$was then created. This matrix is required to calculate the fundamental matrix, $(I - N)^{-1}$, which is central to determining the expected number of visits to each transient state before reaching the absorbing state.

```
I = np.eye(8)
print(I)
print(I.shape)
```

The output was:

```
[[1. 0. 0. 0. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0. 0. 0. 0.]
 [0. 0. 1. 0. 0. 0. 0. 0.]
 [0. 0. 0. 1. 0. 0. 0. 0.]
 [0. 0. 0. 0. 1. 0. 0. 0.]
 [0. 0. 0. 0. 0. 1. 0. 0.]
```

```
 [0. 0. 0. 0. 0. 0. 1. 0.]
 [0. 0. 0. 0. 0. 0. 0. 1.]]
(8, 8)
```

The fundamental matrix was then calculated using

$$F = (I - N)^{-1}$$

where I denotes the identity matrix. The fundamental matrix provides the expected number of visits to each transient state before absorption, making it possible to estimate the expected number of coin flips required to complete the game from any starting position.

```
D= I-N
print(D)
F = np.linalg.inv(D)
print(F)
```

The resulting fundamental matrix is shown below.

```
[ [ 1. 0. 0. -0.5 -0.5 0. 0. 0. ]
[ 0. 1. 0. 0. 0. 0. 0. 0. ]
[ 0. 0. 1. 0. 0. 0. 0. 0. ]
[ 0. -0.5 -0.5 1. 0. -0.5 -0.5 0. ]
[-0.5 -0.5 -0.5 -0.5 1. 0. 0. 0. ]
[ 0. 0. 0. 0. 0. 1. 0. 0. ]
[-0.5 0. 0. 0. -0.5 -0.5 1. 0. ]
[ 0. 0. 0. 0. 0. 0. 0. 1. ] ]
```

```
[[2.33333333, 1.83333333, 1.83333333, 2. 1.66666667 1.5 1. 0. ]
[0. 1. 0. 0. 0. 0. 0. 0. ]
[0. 0. 1. 0. 0. 0. 0. 0. ]
[1. 1.5 1.5 2. 1. 1.5 1. 0. ]
```

```
[1.66666667 2.16666667 2.16666667 2 2.33333333 1.5 1. 0. ]
[0. 0. 0. 0. 0. 1. 0. 0. ]
[2. 2. 2. 2. 2. 2. 2. 0. ]
[0. 0. 0. 0. 0. 0. 0. 1. ]]
```

The values within the fundamental matrix were subsequently summed to determine the expected number of coin flips required to reach the absorbing state from each starting position GeeksforGeeks (2019) :

```
F = np.array(F)
sum_of_columns = np.sum(F,axis=0).tolist()
print(sum_of_columns)
```

The output was:

```
[7.0, 8.5, 8.5, 8.0, 7.0, 7.5, 5.0, 1.0]
```

The calculated expected absorption times closely reflect the structure of the Snakes and Ladders board. As expected, starting from square 7 results in the lowest expected number of coin flips because the absorbing state (square 8) can be reached in a single move regardless of the outcome of the coin flip. This confirms that expected game duration generally decreases as the starting position approaches the terminal state.

Interestingly, square 6 also exhibits a relatively low expected number of coin flips despite being adjacent to the head of a snake. Although the snake introduces the possibility of moving backwards, the square's close proximity to the absorbing state outweighs this disadvantage, illustrating how overall transition probabilities influence expected absorption time.

Squares 0 and 4 require a similar number of expected coin flips despite being positioned differently on the board. Although square 0 represents the furthest point from the absorbing state, it is less disadvantageous than might initially be expected because the ladder located on square 1 provides a relatively high probability of rapidly progressing to square 6. Consequently, the expected game duration is considerably reduced despite the greater starting

distance from the terminal state. Similarly, square 4 represents the midpoint of the board and has comparatively few obstacles before the absorbing state, making it another favourable starting position.

In contrast, squares 1 and 2 produce the highest expected number of coin flips. Although these squares contain ladders, both ultimately lead to positions immediately preceding snakes, increasing the likelihood of regression within the Markov process. Consequently, positions that initially appear advantageous become statistically less favourable due to the structure of subsequent transitions.

1.3 The probability of completing the game without slipping back to square 0

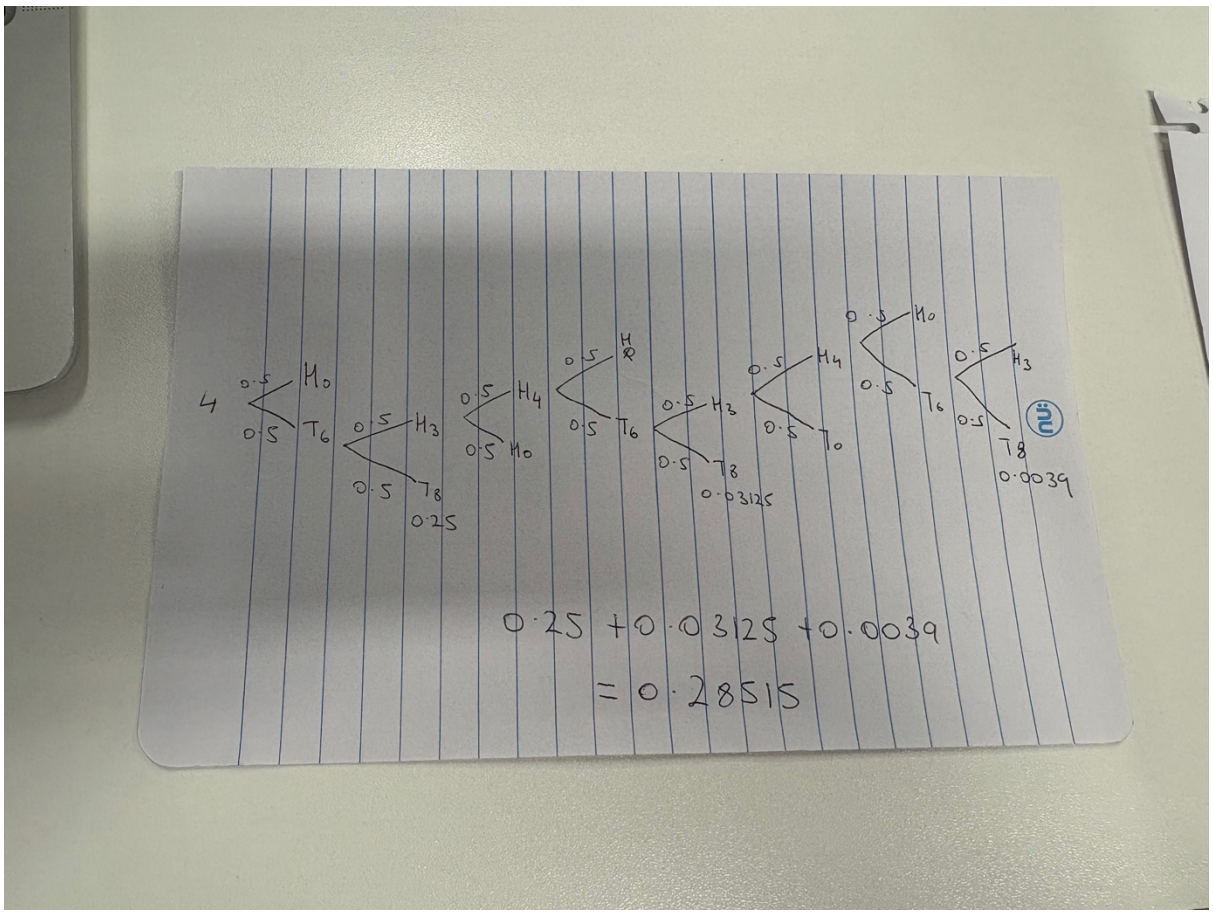
To calculate the probability of completing the game without returning to square 0, a probability tree was constructed beginning from the middle square (square 4). The tree systematically represents every possible transition while excluding any paths that return to square 0, allowing only successful routes to contribute to the final probability. Each branch of the probability tree has an associated probability of 0.5 because the coin is assumed to be fair, giving equal probability to heads and tails.

Starting from square 4, two possible transitions exist: moving directly to square 0, which immediately fails the condition of the problem, or progressing to square 6. From square 6 the player either reaches the absorbing state (square 8) or is redirected to square 3 following the snake on square 7. Consequently, only pathways that eventually reach square 8 without revisiting square 0 contribute to the required probability.

The probability tree naturally develops a repeating cycle between squares 3, 4 and 6, reflecting the recursive behaviour of the Markov Chain. This recursive behaviour is characteristic of Markov processes in which transient states may be revisited multiple times before absorption occurs. This cyclic structure means that multiple successful pathways contribute to the overall probability of reaching the absorbing state, requiring successive probabilities to be accumulated. Summing these successful pathways produced a final probability of approximately 0.285, indicating that a player beginning from the middle square has roughly a 28.5% chance of completing the game without returning to square 0.

This analysis illustrates the Markov property, whereby the probability of the next move depends solely on the player's current position rather than the sequence of previous moves. Once the player reaches square 4, the probability of all future transitions is determined entirely by the transition probabilities associated with that state, demonstrating the memoryless nature of a Markov Chain

Image 1



Conclusion

This investigation demonstrated how absorbing Markov Chains can be used to model stochastic systems and analyse expected behaviour. By constructing the transition matrix, calculating the fundamental matrix and evaluating absorption probabilities, the behaviour of the Snakes and Ladders game could be quantified mathematically. Beyond this specific example, the project illustrates how probabilistic models can be applied to understand more complex real-world systems governed by uncertainty.

Reference:

GeeksforGeeks. (2019, December 13). *Summation Matrix columns Python*. GeeksforGeeks.

https://www.geeksforgeeks.org/python/python-summation-matrix-columns/